Day 122 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 4 reference

Watch: Clustering (4): Gaussian Mixture Models and EM (Alexander Ihler) | Gaussian Mixture Models (GMM) & EM Algorithm Explained Visually (PhyLosophy)

`day122_gmm_em.py`, segment by segment, in the order `main()` uses things. The density, log-sum-exp, the E and M steps, EM itself and BIC are new; Day 119's k-means starts EM, and Day 121's shapes are reused. scikit-learn's GaussianMixture appears only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: w = weights · M = means, one row per cloud · S = covariances · R = responsibilities, one row per point · L = a Cholesky factor, $S = LL^T$ · ll = log-likelihood

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day119_k_means as d119
    import day121_dbscan as d121
ari, standardise = d119.ari, d119.standardise
LOG2PI = np.log(2 * np.pi)
```

- `import day121_dbscan as d121` — importing Day 121 also imports Day 119 through it; both are quiet.
- `LOG2PI = np.log(2 * np.pi)` — computed once and reused in every density.

Segment 1 — The mixture

log_gauss and logsumexp

MATH

$$\log N(x \mid \mu, S) = -\tfrac{1}{2}|z|^2 - \Sigma \log L_{ii} - (d/2)\log 2\pi, \quad L\,z = x - \mu$$

$$\log \Sigma\, e^{a_j} = m + \log \Sigma\, e^{a_j - m}, \quad m = \max a_j$$

CODE

```
def log_gauss(X, mu, S):
    L = np.linalg.cholesky(S)
    Z = np.linalg.solve(L, (X - mu).T)
    return -0.5 * (Z ** 2).sum(0) - np.log(np.diag(L)).sum() - 0.5 * X.shape[1] * LOG2PI
def logsumexp(A):
    m = A.max(1, keepdims=True)
    return (m + np.log(np.exp(A - m).sum(1, keepdims=True)))[:, 0]
```

- `np.linalg.cholesky(S)` — a lower-triangular L with $S = L\,L^T$; it exists only if S is positive definite.
- `np.linalg.solve(L, (X - mu).T)` — solves $L\,z = x - \mu$ for every point at once, without inverting S.
- `np.log(np.diag(L)).sum()` — half of log det S: the determinant of a triangular matrix is the product of its diagonal.

- `m + np.log(np.exp(A - m).sum(...))` — subtracting the largest value first keeps exp from overflowing.

e_step and m_step

MATH

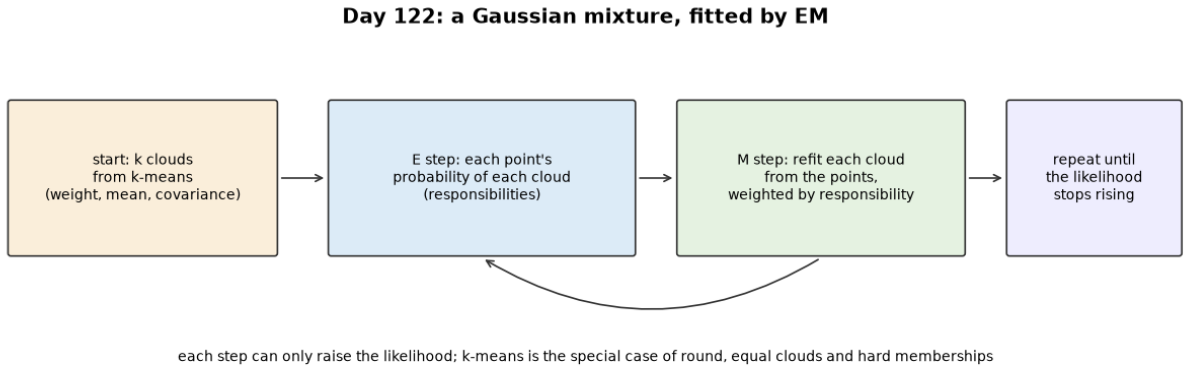
$$r_{ij} = exp(log\ w_j + log\ N_{ij} - log\ p(x_i)); \quad S_j = \Sigma_i\ r_{ij}(x_i - \mu_j)(x_i - \mu_j)^T/N_j + ridge \times I$$

CODE

```
def e_step(X, w, M, S):
    logp = np.stack([np.log(w[j]) + log_gauss(X, M[j], S[j]) for j in range(len(w))], 1)
    ll = logsumexp(logp)
    return np.exp(logp - ll[:, None]), float(ll.sum())
def m_step(X, R, kind="full", reg=1e-6):
    Nk = R.sum(0) + 1e-300; w = Nk / len(X); M = (R.T @ X) / Nk[:, None]; d = X.shape[1]; S = []
    for j in range(R.shape[1]):
        D = X - M[j]; Sj = (R[:, j, None] * D).T @ D / Nk[j]
        if kind == "diag":
            Sj = np.diag(np.diag(Sj))
        elif kind == "spherical":
            Sj = np.eye(d) * np.trace(Sj) / d
        S.append(Sj + reg * np.eye(d))
    return w, M, np.array(S)
```

- `np.exp(logp - ll[:, None])` — responsibilities: each row's log-probabilities minus its total, exponentiated; every row sums to 1.
- `R.T @ X / Nk[:, None]` — every cloud's weighted mean at once.
- `(R[:, j, None] * D).T @ D / Nk[j]` — a weighted covariance: each row of D scaled by its responsibility before the product.
- `np.diag(np.diag(Sj))` — np.diag of a matrix gives its diagonal; of a vector, a diagonal matrix: together they zero the off-diagonal.

SYSTEM DIAGRAM



The EM loop.

gmm, n_params and bic

MATH

$$BIC = -2 \log L + p \log n$$

CODE

```
def gmm(X, k, seed, kind="full", reg=1e-6, iters=500, tol=1e-6):
    lab = d119.kmeans(X, k, seed)[0]; R = np.eye(k)[lab]
    w, M, S = m_step(X, R, kind, reg); history = []
    for _ in range(iters):
        R, ll = e_step(X, w, M, S); history.append(ll)
        if len(history) > 1 and history[-1] - history[-2] < tol * len(X):
            break
        w, M, S = m_step(X, R, kind, reg)
    return {"w": w, "M": M, "S": S, "R": R, "history": history, "init": lab}
def n_params(k, d, kind):
    cov = {"full": d * (d + 1) // 2, "diag": d, "spherical": 1}[kind]
    return (k - 1) + k * d + k * cov
def bic(X, model, kind):
    k, d = model["M"].shape
    return -2 * model["history"][-1] + n_params(k, d, kind) * np.log(len(X))
```

- `np.eye(k)[lab]` — one-hot rows: k-means' hard labels as responsibilities of 0 and 1.
- `history[-1] - history[-2] < tol * len(X)` — stop when the mean log-likelihood rises by less than tol.
- `{"full": ..., "diag": d, "spherical": 1}[kind]` — a dictionary looked up at once: the parameters per covariance.
- `return {"w": w, ...}` — the model as a dictionary of named parts.

Segment 2 — The experiments

match: naming clusters by their majority class

CODE

```
def match(lab, y):
    m = np.array([np.bincount(y[lab == c], minlength=y.max() + 1).argmax() if np.any(lab == c)
        else -1 for c in range(lab.max() + 1)])
    return m[lab] == y
```

- `np.bincount(..., minlength=y.max() + 1).argmax()` — the most common true class in each cluster.
- `m[lab] == y` — each point's cluster name compared with its own class: True where it is right.

demo_check: the same start for both versions

CODE

```
X, y = sets["iris"]; lab = d119.kmeans(X, 3, 0)[0]; w0, M0, S0 = m_step(X, np.eye(3)[lab])
ours = gmm(X, 3, 0, tol=1e-10, iters=2000)
sk = GaussianMixture(3, covariance_type="full", reg_covar=1e-6, weights_init=w0, means_init=M0,
    precisions_init=np.linalg.inv(S0),
                     tol=1e-10, max_iter=2000).fit(X)
```

- `precisions_init=np.linalg.inv(S0)` — scikit-learn takes the inverse covariances to start.
- `tol=1e-10` — both run almost to the exact optimum, so differences come from arithmetic, not stopping rules.

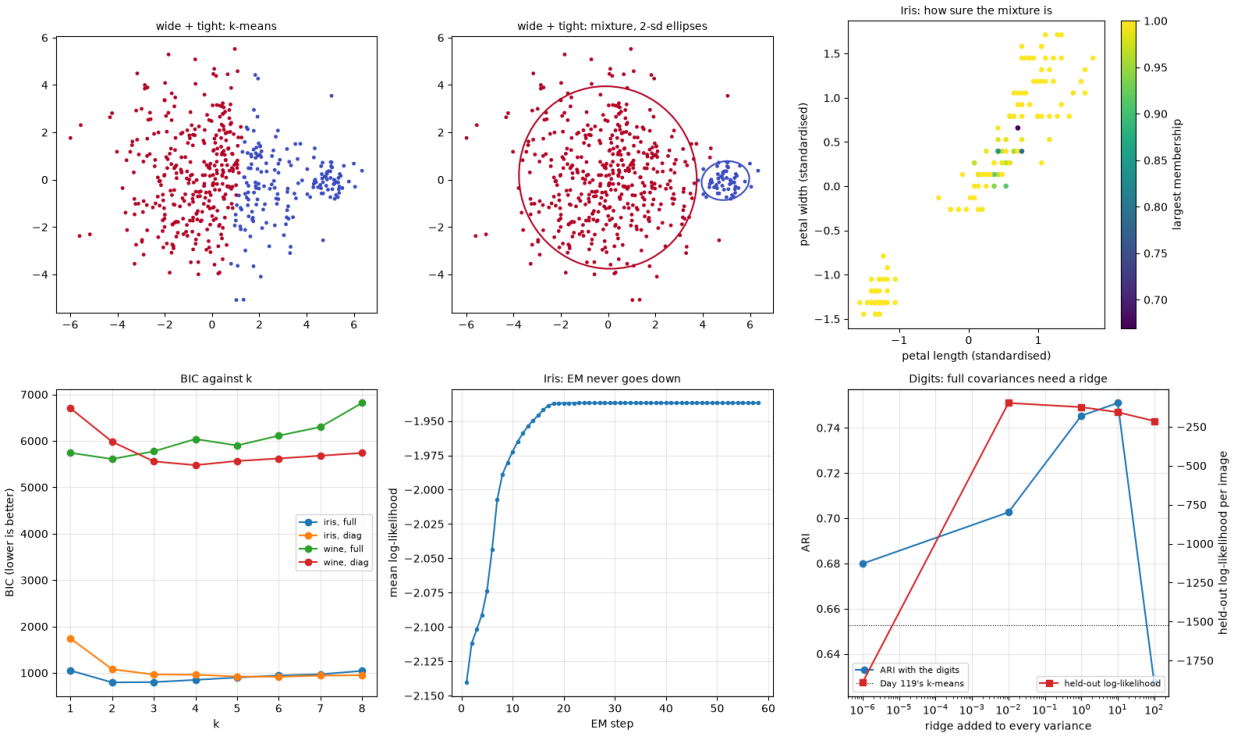
demo_digits: catching a failure, and a held-out split

CODE

```
try:
    gmm(X, 10, 0, "full", reg=0.0)
except np.linalg.LinAlgError as e:
    print(f"  ridge 0: the Cholesky factor fails ('{e}')")
tr = np.random.RandomState(0).permutation(len(X)); te, tr = tr[:540], tr[540:]
print(f"  {'ridge':>7}{'ARI':>8}{'held-out log-likelihood per image':>36}")
```

- `try: ... except np.linalg.LinAlgError as e:` — the failure is caught and printed, so the lesson goes on.
- `te, tr = tr[:540], tr[540:]` — both names are reassigned from the old tr in one line: 540 held out, the rest for fitting.

REAL GRAPH



Real graph from the script: k-means against the mixture, soft memberships, BIC, EM's progress and the ridge.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.cholesky(S)` — S = L L^T, L lower-triangular.
- `np.linalg.solve(L, B)` — solve L X = B without an inverse.
- `np.diag(np.diag(S))` — keep only the diagonal.
- `np.eye(k)[lab]` — one-hot rows.
- `try / except Error as e` — catch a failure and keep its message.
- `Ellipse(mu, w, h, angle=a)` — matplotlib's ellipse patch.

End of Day 122 syntax reference. For the experiments and what they showed, see the Day 122 complete guide.